Software Requirements Specification

Field	This example
Team name	Dek70
Project title	NUDTEE
SDLC chosen + reason	Agile (Scrum, feature-based sprints) — a 5-person team can divide work by functional requirement (FR-1 to FR-7), build and integrate in short iterative sprints, and adjust scope quickly given the limited build window.
Date / version	V2.0 · 2026-11-02
GitHub repo	https://github.com/6931503074-sudo/NUDTEE

1. Introduction

1.1 Purpose and scope

People organizing general group activities currently face significant friction due to unstructured communication in LINE or Facebook group chats. Essential session details such as exact dates, times, pinned locations, and participant limits frequently get buried under long chat threads, causing confusion, uncoordinated schedules, and forgotten appointments. Furthermore, organizers lack a reliable, live method to track open spots, forcing them to manually ask members and recount RSVPs repeatedly in chat threads. This disorganization wastes time and often causes sessions to fall apart despite having enough interested participants.

NUDTEE is a web application designed to streamline general activity scheduling and headcount tracking in real time. It replaces manual chat counting by providing a centralized platform featuring interactive map location pinning, live seat count synchronization, automatic session closing at specified deadlines, in-room real-time chat, and seamless export to Google Calendar. Success looks like this: an organizer creates a session with a pinned location and deadline, participants join with 2 taps or fewer while seeing instant real-time headcount updates, and members sync their joined sessions directly to Google Calendar to prevent forgotten appointments.

1.2 Users and stakeholders

Name / role	Kind	What they need	What they fear / forbid
Player/Participant(primary user)	primary user	Find open activity sessions with clear dates, times, and locations; view real-time available spots; join with 2 taps or fewer; communicate via in-room chat; and add joined events to Google Calendar.	Joining sessions that are already full or closed; missing or forgetting activity schedules due to disorganized chat messages; getting lost due to unclear location details.
Session Organizer (Primary user)	primary user	Organize group activities with explicit participant limits, automatic closing at a registration deadline, and a chosen map location (or online meeting link) without manually asking or recounting RSVPs in chat threads.	Ending up short of participants; dealing with manual headcount tracking; having participants forget or miss the session schedule.
Hosting & Service Providers (Google Firebase, Google Maps and Calendar, OpenStreetMap-based	Supplier / External provider	Database connections, sign-in, map and search requests that operate strictly within free-tier quotas and usage limits	Excessive traffic spikes, unoptimized real-time listeners, or API misuse that exceeds free-tier limits or triggers paid plans.

Name / role	Kind	What they need	What they fear / forbid
place search, Vercel/Netlify)		throughout the project build window.	

2. Overall description

2.1 Product context

Priority	Items
Must (MVP core)	FR-1: Create a session (title, location or online meeting link, date/time, min/max players, registration deadline, category) FR-2: Join / leave a session FR-3: Real-time session status display FR-4: Display location on map FR-6: Auto-close session at the registration deadline FR-5: In-room chat FR-7: Add to Google Calendar
Should	-
Could (implemented as lightweight extras)	Category list and category/sub-category filter in the session list, and category selection when creating a session · User profile page (avatar, display name, username, bio) · Session cover image · Online sessions with a meeting link · Time-overlap check against sessions the user has already joined · Organizer tools: remove a participant, close the room · “Use my current location” button when choosing a location · Share-link button · Language menu (Thai and English supported, further languages partially translated). These are not covered by dedicated FRs or acceptance tests this term.

Priority	Items
Won’t (this term,explicit promise)	Player rating or review system, venue/cafe booking & payment integration, friend system / 1-on-1 direct messaging outside session chat rooms · Editing a session after creation (the organizer closes the room and creates a new one) · Map overview showing pins of all sessions (a location is viewed per session)

2.2 Assumptions and constraints

Assumptions

- Device & Connectivity: Every user has access to a smartphone or laptop with a stable internet connection and a modern web browser.
- Authentication & Services: Users are willing to sign in with a Google account (through Firebase Authentication) and to open Google Calendar in the browser to save an event.
- Location Access: Users can search for a place by name. Using “Use my current location” requires the user to allow the browser’s location permission; it is optional.

Constraints

- Backend & Database: Backend infrastructure is constrained to the Google Firebase free (Spark) plan — Firebase Authentication and Cloud Firestore with real-time listeners — and must stay within its free quotas.

- Third-Party Services: All integrated services must operate within free usage and incur zero financial cost: Google sign-in (Firebase Authentication), the embedded Google Maps view (no API key), the Google Calendar event link, and the OpenStreetMap-based place search (public Nominatim and Photon services, used lightly and respecting their usage policies).

- Hosting: Frontend deployment is limited to the free tier of Vercel or Netlify (static web page).

- Localization: Thai and English are the supported UI languages. The language menu also offers eight further languages with partial translations of core labels; untranslated text falls back to English, and these languages are not tested.

- Data Privacy: Stored data is limited to: Google account ID and display name; optional profile fields the user enters (username, short bio, small profile-picture thumbnail); session details (title, descriptions, category, online/offline type, location name, address and coordinates or meeting link, date, start/end time, registration deadline, player limits, resized cover image); participation records; and in-room chat messages (sender ID, name, text, timestamp). The user’s own full profile is kept in

the browser’s local storage. No payment data, private 1-on-1 direct messages, or sensitive personal information is stored.

- Resource Constraints: Project development is limited to a 5-person undergraduate team within a shared academic semester build window.

3. Functional requirements

FR-1 Create

User story:

As a session organizer, I want to create an activity session with a title, location (or online meeting link), date/time, minimum/maximum players, registration deadline and category, so that others can view structured details and decide whether to join.

Pain this traces to:

Activity announcements are currently made informally in chat threads without a structured place to view exact details, schedules, or locations.

Acceptance Criteria:

- Given a title, event date, start and end time, registration deadline, category, min/max players, and a location chosen from the place-search results or from the device's current location, when the organizer saves the session, then the session appears in the session list under its category and its details page shows those details.
- Given the organizer selects "Online", then no map location is required, and an optional meeting link/platform is shown on the session instead of a map.
- Given a session is saved successfully, then the organizer is automatically added as the first participant and counts toward the maximum and minimum number of players.

Error: Given a required field is empty (title, event date, registration deadline, start time, end time, or the location of an offline session), when saving, then the request is rejected and an error message appears on the form.

Error: Given a typed location that was not chosen from the search results, when saving, then it is rejected with "Pick a real place from the search results."

Error: Given min players is below 1, or max players is below min players, when saving, then it is rejected with an error message.

Error: Given the registration deadline is after the event date, the end time is not after the start time, or the end time is in the past, when saving, then it is rejected with an error message.

Error: Given the session's time overlaps with another session the organizer has already joined, when saving, then it is rejected with a message naming the conflicting session.

Decision logic in this FR: The registration deadline is a calendar date; registration closes at the end of that day or when the session starts, whichever comes first (see FR-6). To change a session, the organizer closes the room and creates a new one (editing is out of scope).

FR-2 Join / leave a session

User story: As a participant, I want to join or leave a session with 2 taps or fewer, so that the organizer does not need to manually count replies in chat.

Pain this traces to: Organizers currently have no reliable running headcount and must repeatedly ask around in chat threads.

Acceptance Criteria:

- Given an open session with available spots before registration closes, when a user taps Join, then they are added to the participant list and remaining spots decrease by 1.
- Given a participant who is not the organizer, in a session that has not ended, when they tap Leave, then they are removed from the participant list and remaining spots increase by 1.

Error: Given a session is full (current == max), when a user attempts to join, then the request is rejected and “This room is full.” is displayed.

Error: Given a user who has already joined, then the session shows “Joined” and the user cannot be added to the participant list a second time.

Error: Given two users tap Join simultaneously with only 1 spot left, then exactly 1 user succeeds and the other receives “This room is full.”

Error: Given the session’s time overlaps with another session the user has already joined, when they tap Join, then the request is rejected with a message naming the conflicting session.

Decision logic in this FR: The system computes remaining_spots = max_players – current_joined. Join and leave run as Cloud Firestore transactions that re-read the participant list and reject the join when it is already full, so remaining spots never go negative.

FR-3 Real-time session status display

User story: As a user or organizer, I want session details and participant counts to update live as people join or leave, so that headcount information is always accurate without refreshing.

Pain this traces to: Headcounts discussed in chat threads quickly become stale as people change their minds, forcing repeated manual recounts.

Acceptance Criteria:

- Given another user joins, leaves, creates, or closes a session, when the change is saved, then all connected users viewing the list see the updated count within 2 seconds without refreshing.
- Given a session becomes full or its registration deadline has passed, then its status shows “Full” or “Closed” (the statuses are Open, Almost Full, Full, Joined, and Closed). “Closed” is evaluated whenever the list or details are displayed.

Error: Given a client loses connection, then when the connection returns the real-time listeners resume automatically and the displayed data resynchronizes. A dedicated “reconnecting” indicator is not part of this release.

Decision logic in this FR: Client state is synchronized through Cloud Firestore real-time listeners (snapshot subscriptions) on the session and chat data.

FR-4 Display location on map

User story: As a participant, I want to view a session’s location on an interactive map, so that I can easily navigate to the venue without getting lost.

Pain this traces to: Vague text descriptions in chat threads cause participants to arrive at wrong locations or struggle with navigation.

Acceptance Criteria:

- Given an offline session with a saved location, when a user opens “View Location” or the map preview on the session details, then an interactive map centered on the session’s coordinates is displayed with pan and zoom controls, together with an “Open in Google Maps” link.
- Given a session’s details, then the location name and full address are also shown as text.

Error / Empty: Given an online session, then no map is shown and the meeting link/platform is displayed instead.

FR-5 In-room chat

User story: As a joined participant, I want to send and receive real-time messages in a dedicated session chat room, so that I can coordinate logistical details with attendees.

Pain this traces to: Event logistics get lost or mixed up when discussed across multiple general chat groups.

Acceptance Criteria:

- Given a user who has joined a session, when they open the session chat room, then they can send and receive messages in real time.
- Given a message is sent, then all participants in that room receive the message within 2 seconds.
- Given a session has ended, then its chat room is read-only.

Error: Given a user who is not a participant, when they attempt to open or send messages in the chat room, then access is denied ("Join this room to use its chat.").

.

FR-6 Auto-close session

User story: As an organizer and participant, I want the system to automatically block new joins once the registration deadline is reached, so that last-minute additions do not disrupt activity plans.

Pain this traces to: Participants joining after finalized preparations cause headcount mismatches.

Acceptance Criteria:

- Given a session reaches the end of its registration deadline day, or its start time (whichever comes first), then the session is shown as "Closed" and joining is disabled.
- Given a session's end time has passed, then the session is marked as completed, removed from the session list, and remains visible to its participants under My Activities ⟶ Completed.

Error: Given a join attempt after registration has closed, then the request is rejected with "Registration is closed for this room." (100% rejection rate for attempts made through the application).

Decision logic in this FR: Timestamp validation (current time compared with the end of the deadline day and the session start time) is executed in the join flow before the join transaction is processed.

FR-7 Add to Google Calendar

User story: As a participant, I want to add my joined session to my Google Calendar, so that I receive schedule reminders and avoid forgetting appointments.

Pain this traces to: Participants frequently forget activity dates and times when schedules exist only in chat threads.

Acceptance Criteria:

- Given a joined session, when the participant taps "Add to Calendar", then Google Calendar opens with the event (title, start/end time, location or meeting link, description) pre-filled, and the participant saves it with one more tap (2 taps or fewer in total). Signing in to Google, if needed, is not counted.

Error: Given the participant is not signed in to Google or cancels, then no event is created and the participant can tap "Add to Calendar" again.

Decision logic in this FR: The event is created through Google Calendar's event-template link (times in Asia/Bangkok), so no OAuth access token or Calendar API call is handled by NUDTEE.

INVEST check

FR	Independent	Negotiable	Valuable	Estimable	Small	Testable	Note
FR-1	Yes	Yes	Yes	Yes	Yes	Yes	Core prerequisite for all session data; can be built and tested independently using standard form validation.
FR-2	Partial	Yes	Yes	Yes	Yes	Yes	Depends on FR-1 (needs existing sessions); can be tested with seeded session records.
FR-3	Partial	Yes	Yes	Yes	Partial	Yes	Depends on FR-2 state changes; requires testing of Firestore real-time listeners across concurrent client devices.
FR-4	Partial	Yes	Yes	Yes	Yes	Yes	Reads the location

							coordinates saved by FR-1.
FR-5	Partial	Yes	Yes	Yes	Partial	Yes	Depends on FR-2 (the participant list decides who may use the chat); relies on real-time messaging.
FR-6	Partial	Yes	Yes	Yes	Yes	Yes	Depends on FR-1 (registration deadline) and is enforced inside the FR-2 join flow before a join is processed.
FR-7	Partial	Yes	Yes	Yes	Yes	Yes	Depends on FR-2 (joined state) and FR-1 (time, location); the calendar link can be built and tested with a mock joined session.

4. Non-functional requirements

ID	Kind	Statement (testable)	How we will check
NFR-1	performance	The session list shall reflect a join/leave change, and in-room chat messages shall be delivered, to all connected clients within 2 seconds.	Two devices open the same session/chat; one joins/sends a message, time until the other updates/receives it.
NFR-2	usability	The app shall be usable on a mobile phone screen without horizontal scrolling on any core screen. Additionally, adding a session to Google Calendar, and joining an open session from the session list, shall each take 2 taps or fewer (excluding Google sign-in).	Manual check on a real phone for each of the core screens; manually count the taps required for the calendar export and for joining from the list.
NFR-3	capacity	The system shall support at least 20 concurrently connected users viewing the real-time session list and sending chat messages without errors.	Load test with 20 simulated concurrent connections interacting with real-time listeners.
NFR-4	reliability	A session’s remaining-spot count shall never go negative or exceed the stated maximum. Furthermore, 100% of join attempts submitted through the application after registration has closed shall be rejected.	Two clients tap Join simultaneously on a session with one spot left; verify only one succeeds. Attempt to join a closed session and verify immediate rejection.

ID	Kind	Statement (testable)	How we will check
NFR-5	security / privacy (required)	A user shall be able to edit or close only sessions they created. Additionally, a user cannot access or read the in-room chat unless they are a participant. These rules shall be enforced by the database security rules, not only by the user interface.	Attempt to modify another account's session directly through the database API and confirm it is rejected. Attempt to fetch chat messages of an unjoined session and confirm denial.
NFR-6	usability	An organizer shall be able to create a session with all required fields in 60 seconds or less.	Time 3 test users creating a session from the home screen with a stopwatch.

5. Use cases

5.1 Use-case list

Use case	Actor	Related FR	Brief success story
UC-1 Create session	Organizer	FR-1	Organizer creates an activity session with a title, location (or online meeting link), date/time, player limits, registration deadline and category; it immediately appears on the session list.
UC-2: Join / leave session	Participant	FR-2, FR-6	Participant joins an open session before registration closes, decreasing the remaining spot count by 1, or leaves a joined session to free up a spot.
UC-3: View live session status	Participant, Organizer	FR-3	Users view the session list; player counts and session statuses (Open, Almost Full, Full, Joined, Closed) update in real time without page refreshes.
UC-4: View location on map	Participant	FR-4	Participant views the location of a session on an interactive map to get navigation details.
UC-5: Participate in room chat	Participant, Organizer	FR-5	Session participants send and receive real-time messages inside the session's dedicated chat room.
UC-6: Add to Google Calendar	Participant, Organizer	FR-7	User adds a joined session to Google Calendar in 2 taps or

Use case	Actor	Related FR	Brief success story
			fewer (excluding Google sign-in) to set up schedule reminders.

6. Out of scope

1. Waitlist Management: Managing an automated queue or waitlist for users attempting to join a session that is already full.

2. Push Notifications & Native Reminders: Sending in-app push notifications, SMS, or email alerts (schedule reminders are handled entirely via the external Google Calendar).

3. User Ratings and Reviews: Rating or reviewing session organizers, participants, or venue locations.

4. Venue Booking and Payment Integration: Direct booking mechanisms with commercial venues or cafes, as well as in-app payment processing or bill splitting.

5. Social & Direct Messaging System: Friend list management, user profile following, social feed features, and private 1-on-1 direct messaging outside of session-specific chat rooms.

6. Editing a Session After Creation: The organizer closes the room and creates a new one instead.

7. Map Overview of All Sessions: Pins and summary cards for all sessions on one map (a location is viewed per session).

8. Calendar API Integration: Handling Google OAuth access tokens or Calendar API calls inside NUDTEE (the export uses a pre-filled Google Calendar event link).

9. **7. Traceability matrix (Golden Thread)**

Problem / pain	Requirement (FR)	Solution / design	Feature built	Test
No structured place to announce activities and essential details/locations get buried in chat threads.	FR-1: Create / edit session	Session form with place search and current-location picker (OpenStreetMap-based search), online option, player limits, and registration deadline validation; session stored in Cloud Firestore.	Create Session form with location selection, category, and registration deadline controls.	later
No reliable running headcount, forcing organizers to ask and count manually in chat.	FR-2: Join / leave session	Cloud Firestore transaction for join/leave with capacity checking.	Join/leave button with live attendee count (current/max).	later
Headcount counts go stale between chat messages as people change their minds.	FR-3: Real-time session status	Cloud Firestore real-time listeners on session records.	Live-updating session list with status indicators (Open, Almost Full, Full,	later

Problem / pain	Requirement (FR)	Solution / design	Feature built	Test
			Joined, Closed).	
Unclear activity locations causing participants to get lost or arrive late.	FR-4: Display location on map	Embedded Google Maps view centered on the saved coordinates, with an “Open in Google Maps” link.	View Location screen and map preview on the session details.	later
Disorganized logistical coordination across general group chats.	FR-5: In-room chat	Per-session message collection in Cloud Firestore, opened only for participants.	Room Chat for joined attendees.	later
Uncontrolled last-minute joins after event preparations are finalized.	FR-6: Auto-close session	Time check (deadline day end and session start) in the join flow before the join transaction.	Session shown as “Closed” and late join attempts rejected; ended sessions marked completed.	later
Participants forgetting scheduled activities due to	FR-7: Add to Google Calendar	Google Calendar event-template link pre-filled with the session details.	“Add to Calendar” button on the session details.	later

Problem / pain	Requirement (FR)	Solution / design	Feature built	Test
lack of reminders.				